

Подготовка приложения к Production-окружению

Конфигурация, health checks, graceful shutdown, reverse proxy и production readiness

Результаты занятия

- ✓ Объяснять различия development и production environment
- ✓ Хранить конфигурацию вне исходного кода
- ✓ Проверять обязательные environment variables при startup
- ✓ Настраивать NODE_ENV=production
- ✓ Разделять app.js и server.js
- ✓ Реализовывать liveness и readiness endpoints
- ✓ Корректно обрабатывать SIGTERM/SIGINT
- ✓ Закрывать HTTP server и Prisma connections
- ✓ Понимать роль reverse proxy и trust proxy
- ✓ Проводить production-readiness checklist перед deployment

Учебные вопросы

- 1 Production configuration** Environment variables, secrets и startup validation
- 2 Startup приложения** app.js, server.js и fail fast
- 3 Health checks** Liveness и readiness
- 4 Graceful shutdown** SIGTERM/SIGINT, HTTP server и Prisma
- 5 Работа за reverse proxy** HTTPS termination, trust proxy и production checklist



Учебный вопрос 1. Production configuration

Production-приложение должно получать configuration из environment, а не из исходного кода

Development и Production

- Development оптимизирован для удобства разработки и диагностики.
- Production ориентирован на безопасность, стабильность и предсказуемость.
- Подробные stack traces полезны разработчику, но опасны для внешнего клиента.
- Локальный .env удобен в development, но production secrets должны управляться отдельно.
- Один и тот же код приложения должен запускаться с разной configuration.

Development vs Production

Development

- Verbose logs
- Локальный .env
- Hot reload
- Debugging tools
- Тестовые данные

Production

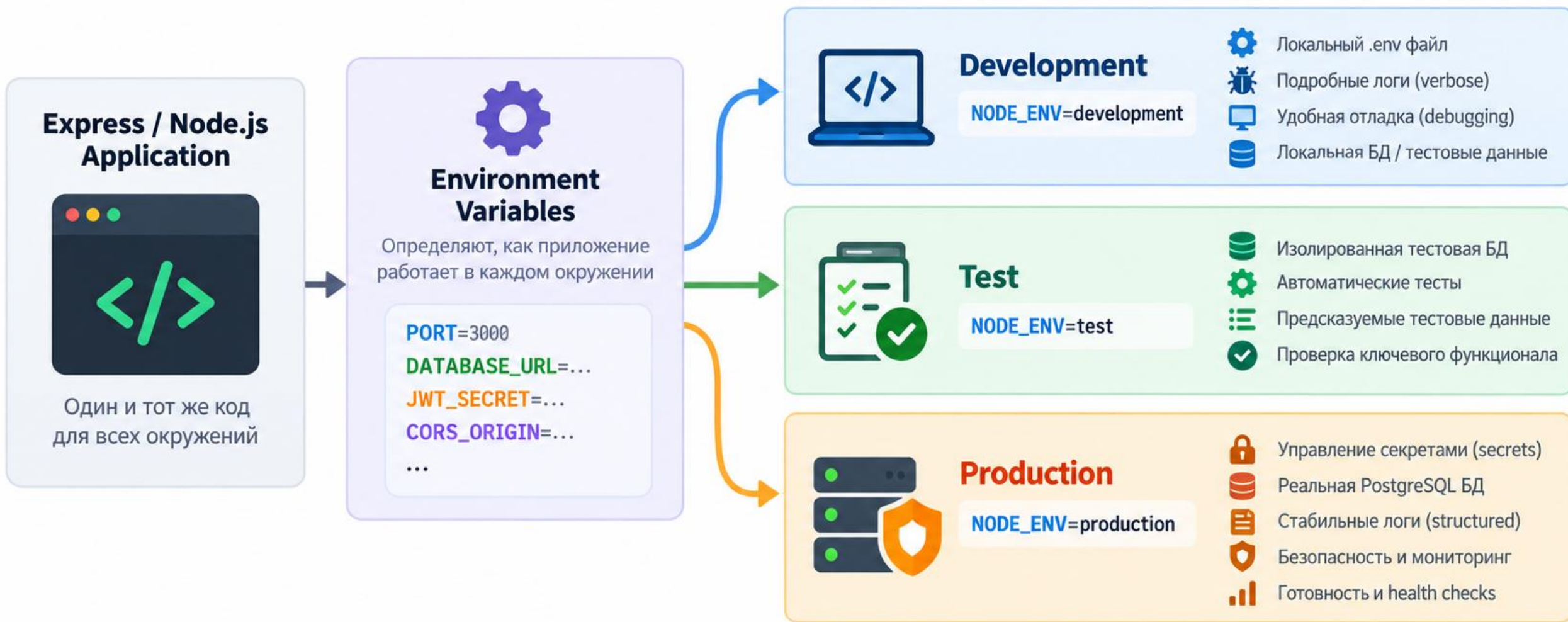
- NODE_ENV=production
- Managed secrets/environment
- Structured logs
- Health checks
- Graceful shutdown

Что относится к configuration

- PORT.
- DATABASE_URL.
- JWT_SECRET.
- CORS allowed origin.
- LOG_LEVEL.
- Trust proxy policy.
- Service URLs внешних систем.
- Feature flags — если проект их использует.

Один код → разные environments

development / test / production через environment variables



Код приложения один и тот же, а поведение и настройки определяются environment variables.

Secrets нельзя хранить в репозитории

- JWT secret не должен быть literal в JavaScript-файле.
- DATABASE_URL может содержать пароль.
- Production .env не должен попадать в Git.
- Не выводите secrets в startup logs.
- Для deployment secrets обычно задаёт ОС, process manager, container platform или secret store.

Пример production environment

TEXT

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://app_user:***@db:5432/shop
JWT_SECRET=***
LOG_LEVEL=info
CORS_ORIGIN=https://shop.example.com
```

process.env возвращает строки

- PORT нужно явно преобразовать в Number.
- Boolean-like values нужно разбирать явно.
- Отсутствующая переменная даёт undefined.
- Ошибку configuration лучше обнаружить при startup, а не при первом HTTP request.

requiredEnv() — fail fast

JAVASCRIPT

```
function requiredEnv(name) {  
  const value = process.env[name];  
  
  if (!value) {  
    throw new Error(`Missing required environment variable: ${name}`);  
  }  
  
  return value;  
}
```

Что важно

- Функция проверяет обязательную variable сразу при startup.
- Приложение не запускается в полунастроенном состоянии.

config/env.js

JAVASCRIPT

```
export const env = {  
  nodeEnv: process.env.NODE_ENV ?? "development",  
  port: Number(process.env.PORT ?? 3000),  
  databaseUrl: requiredEnv("DATABASE_URL"),  
  jwtSecret: requiredEnv("JWT_SECRET"),  
  corsOrigin: requiredEnv("CORS_ORIGIN"),  
  logLevel: process.env.LOG_LEVEL ?? "info"  
};
```

Проверяйте типы configuration

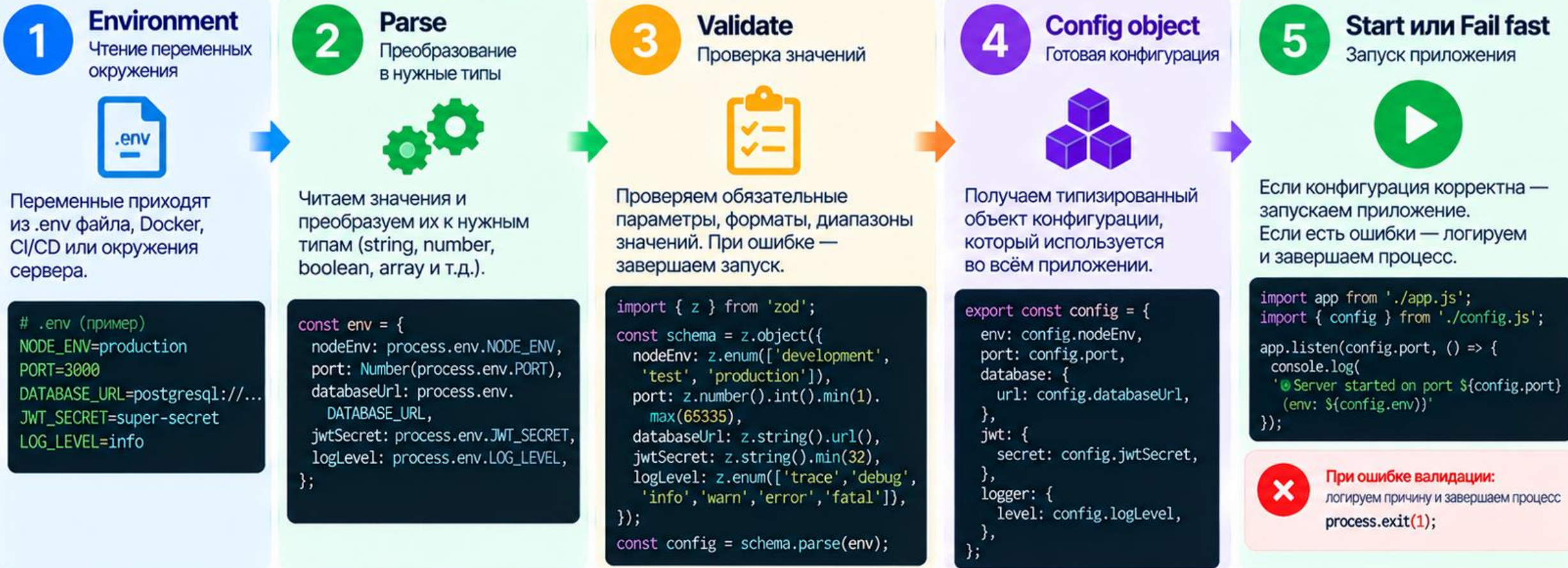
- `Number.isInteger(env.port)`.
- PORT должен быть в допустимом диапазоне.
- CORS_ORIGIN должен быть ожидаемым URL/origin.
- NODE_ENV должен иметь известное значение.
- Ошибка configuration должна быть понятной и завершать startup.

Startup validation: от окружения к запуску

Читаем переменные окружения, парсим и валидируем их, формируем конфигурацию. Если всё корректно — запускаем приложение, иначе — завершаем работу (fail fast).



Fail fast — лучше остановить приложение при некорректной конфигурации, чем запускаться с ошибками и получить проблемы в runtime.



- Преимущества**
- ✓ Раннее обнаружение ошибок конфигурации
 - ✓ Предсказуемое поведение приложения
 - ✓ Безопасный и устойчивый запуск
 - ✓ Одинаковый подход для локальной разработки, тестов и production

- Примеры ошибок валидации**
- Отсутствует обязательная переменная (например, JWT_SECRET)
 - Неверный формат URL базы данных
 - Некорректный порт (выход за диапазон 1–65535)
 - Слишком короткий секретный ключ
 - Недопустимое значение NODE_ENV

Результат

Приложение запускается только с корректной конфигурацией и готово к работе.

Правильная конфигурация — стабильное приложение!



Учебный вопрос 2. Startup приложения

Создание Express app и запуск HTTP server — разные обязанности

app.js и server.js

- app.js создаёт Express application и middleware/routes.
- server.js загружает configuration и запускает HTTP listener.
- Tests импортируют app без постоянного listener.
- Production entry point — server.js.
- Такую архитектуру уже использовали в Supertest integration tests.

app.js

JAVASCRIPT

```
import express from "express";
import { productRoutes } from "../routes/productRoutes.js";

export const app = express();

app.disable("x-powered-by");
app.use(express.json({ limit: "100kb" }));
app.use("/api/products", productRoutes);
```

server.js

JAVASCRIPT

```
import { app } from "./app.js";
import { env } from "./config/env.js";
import { logger } from "./config/logger.js";

export const server = app.listen(env.port, () => {
  logger.info({
    port: env.port,
    nodeEnv: env.nodeEnv
  }, "HTTP server started");
});
```

Что важно

- Если config/env.js выбросит error, listener не стартует.
- Startup log не содержит secret values.

NODE_ENV=production

- Express рекомендует устанавливать NODE_ENV=production в production.
- Environment-specific code можно выбирать через process.env.NODE_ENV.
- Не используйте NODE_ENV как замену полноценной validation configuration.
- Production behavior должен быть явным и проверяемым.

Startup failure — это полезно

- Нет DATABASE_URL → лучше завершить startup.
- Некорректный PORT → лучше завершить startup.
- Нет JWT_SECRET → нельзя принимать login requests.
- Fail fast быстрее обнаруживается process manager/container platform.
- Полуживой процесс сложнее диагностировать и опаснее эксплуатировать.

Production startup: config validation → connect dependencies → listen → ready

1. Config validation

```
.env
NODE_ENV=production
PORT=3000
DATABASE_URL=...
REDIS_URL=...
JWT_SECRET=...
```



parse
(чтение и парсинг)



validate
(проверка по схеме)

Проверяем обязательные переменные:

PORT

DATABASE_URL

REDIS_URL

JWT_SECRET

и другие ...



fail fast при ошибке

Если конфигурация некорректна или отсутствуют обязательные переменные.

2. Connect dependencies



Приложение



PostgreSQL
(база данных)



Redis
(кэш)



внешние сервисы
(очереди, API)



логирование / мониторинг
(например, Sentry, Datadog)



проверка подключения

Проверяем, что все критичные зависимости доступны и работают корректно.

3. Listen

```
const app = express();
const PORT = process.env.PORT;

app.listen(PORT, () => {
  console.log('Server started on port ${PORT}');
});
```



Сервер открывает порт и начинает принимать соединения.

4. Ready



ready
healthcheck OK

```
GET /health

200 OK

{
  "status": "ok",
  "uptime": 12345
}
```



приём запросов

(клиенты, пользователи, другие сервисы)

Некорректная конфигурация
или отсутствуют переменные

Не удалось подключиться
к критичной зависимости



Startup failed

процесс завершён



Сначала проверяем конфигурацию и зависимости, затем открываем порт и только после этого считаем приложение готовым.



Учебный вопрос 3. Health checks

Внешняя система должна понимать: процесс жив и готов ли он обслуживать traffic

Liveness и Readiness

- Liveness: процесс приложения работает.
- Readiness: instance готов принимать traffic.
- Readiness может учитывать критические dependencies.
- Не смешивайте эти две проверки в один смысл.
- Health endpoints должны быть быстрыми и предсказуемыми.

Liveness vs Readiness

Liveness /health/live

- Проверяет сам процесс
- Обычно не делает тяжёлых DB queries
- 200 означает: процесс жив
- Используется для restart decision

Readiness /health/ready

- Проверяет готовность обслуживать requests
- Может проверить DB connection
- 503 означает: временно не направлять traffic
- Используется load balancer/orchestrator

Liveness endpoint

JAVASCRIPT

```
app.get("/health/live", (req, res) => {  
  res.status(200).json({  
    status: "ok"  
  });  
});
```

Readiness endpoint с PostgreSQL

JAVASCRIPT

```
app.get("/health/ready", async (req, res) => {  
  try {  
    await prisma.$queryRaw`SELECT 1`;   
  
    res.status(200).json({  
      status: "ready"  
    });  
  } catch (error) {  
    res.status(503).json({  
      status: "not_ready"  
    });  
  }  
});
```

Что важно

- Readiness проверяет критическую dependency — PostgreSQL.
- При проблеме instance жив, но traffic принимать не должен.

Что не стоит возвращать из health endpoint

- DATABASE_URL.
- JWT secret.
- Stack trace.
- Полную configuration.
- Внутренние credentials.
- Подробности, которые не нужны health checker.

Health model: liveness и readiness



Liveness process alive



GET /health/live

Клиент
(проверка)



```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "status": "alive"
}
```



Процесс жив
(heartbeat OK)



Жив ли процесс?

Liveness не требует, чтобы все зависимости были доступны.



Readiness app + critical dependencies ready



GET /health/ready

Клиент
(проверка)



```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "status": "ready"
}
```



```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
{
  "status": "not ready",
  "reason": "database unavailable"
}
```



Готово ли приложение принимать запросы?

Readiness возвращает 200 OK только если приложение и критические зависимости подключены и работают.

Критерий

Liveness (process alive)

Readiness (app + critical dependencies ready)



Что проверяет?

Проверяет, что процесс не завис и не завершился.

Проверяет, что приложение и критические зависимости готовы.



Отвечает на вопрос

Жив ли процесс?

Готово ли приложение принимать запросы?



Зависимости

Не требует доступности зависимостей.

Проверяет подключение к критическим зависимостям (например, PostgreSQL, Redis, внешние API и т.п.).



Если проверка не пройдена

Перезапустить контейнер / процесс (restart).

Временно не отправлять трафик на приложение (не готово принимать запросы).



Учебный вопрос 4. Graceful shutdown

Production instance должен корректно завершать текущую работу перед остановкой

Что такое graceful shutdown

- Перестать принимать новые connections/requests.
- Дать активным requests завершиться.
- Закрыть HTTP server.
- Закрыть database connections.
- Записать финальные logs.
- Завершить process после cleanup.

SIGTERM и SIGINT

- SIGTERM обычно используется process manager/container platform для корректной остановки.
- SIGINT часто приходит из terminal через Ctrl+C.
- Node.js позволяет установить signal handlers.
- После установки собственного handler нужно самим организовать завершение процесса.

Signal handlers

JAVASCRIPT

```
process.once("SIGTERM", () => {  
  shutdown("SIGTERM");  
});
```

```
process.once("SIGINT", () => {  
  shutdown("SIGINT");  
});
```

Graceful shutdown HTTP + Prisma

JAVASCRIPT

```
let shuttingDown = false;

async function shutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;

  logger.info({ signal }, "Shutdown started");

  server.close(async (error) => {
    if (error) {
      logger.error({ err: error }, "HTTP server close failed");
      process.exitCode = 1;
      return;
    }

    await prisma.$disconnect();
    logger.info("Shutdown completed");
  });
```

Что важно

- `server.close()` перестает принимать новые connections и ждёт завершения активных requests.
- После закрытия HTTP server отключаем Prisma.

Shutdown timeout

- Активный request может зависнуть.
- Нужен максимальный срок ожидания shutdown.
- После timeout можно принудительно закрыть connections.
- Принудительное завершение — fallback, а не основной путь.
- Timeout должен быть согласован с инфраструктурой deployment.

Graceful shutdown: корректное завершение Node.js/Express

Получаем SIGTERM → перестаём принимать новые запросы → ждём завершения текущих → закрываем HTTP-сервер → отключаем Prisma → выходим из процесса.



Graceful shutdown позволяет без потери данных корректно завершить приложение при деплое, масштабировании и остановке контейнера.

1 SIGTERM

Получаем сигнал на завершение



Сигнал отправляет оркестратор (Docker, Kubernetes, системный сервис) при деплое или остановке.

```
process.on('SIGTERM', () => {
  logger.info('SIGTERM received.
  Starting graceful shutdown...');
  shutdown('SIGTERM');
});
```

```
process.on('SIGINT', () => {
  logger.info('SIGINT received.');
```

2 Stop new traffic

Прекращаем принимать новые запросы



Снимаем приложение из балансировки / помечаем как "draining". Новые запросы не поступают.

```
// Пример: флаг, чтобы
// middleware отвергал новые
// запросы
let isShuttingDown = true;

app.use((req, res, next) => {
  if (isShuttingDown) {
    return res.status(503).json({
      error: 'Server is shutting
      down. Please retry later.'
    });
  }
  next();
});
```

3 Finish requests

Дожидаемся завершения текущих запросов



Не прерываем активные запросы. Ждём, пока они завершатся (с таймаутом на всякий случай).

```
const shutdown = async (signal) => {
  logger.info('Graceful shutdown
  started (${signal})');
  // Ставим таймер на принудительное
  // завершение (fallback)
  const timeout = setTimeout(() => {
    logger.error('Forced shutdown
    due to timeout');
    process.exit(1);
  }, 30000); // 30 секунд
  try {
    // дальше шаги 4-5...
  } finally {
    clearTimeout(timeout);
  }
};
```

4 Close HTTP

Закрываем HTTP-сервер



server.close() прекращает приём новых соединений, но позволяет завершить текущие.

```
await new Promise((resolve,
reject) => {
  logger.info('Closing HTTP server...');
  server.close((err) => {
    if (err) return reject(err);
    logger.info('HTTP server closed.');
```

5 Disconnect Prisma

Закрываем подключения к БД



Корректно закрываем соединения с базой данных (Prisma), чтобы избежать утечек и блокировок.

```
try {
  logger.info('Disconnecting
  Prisma...');
  await prisma.$disconnect();
  logger.info('Prisma
  disconnected.');
```

6 Exit

Завершаем процесс



Все ресурсы освобождены. Завершаем процесс с кодом 0 (успешно).

```
logger.info('Graceful
shutdown completed.');
```



Если сработал таймаут или произошла ошибка — завершаем с кодом 1.
`process.exit(1);`



Преимущества

- ✓ Нет потери данных и недозавершённых операций
- ✓ Корректные логи и освобождение ресурсов
- ✓ Быстрые и безопасные деплои (zero downtime)
- ✓ Стабильная работа в Kubernetes/Docker



Рекомендуемые тайминги

- ✓ Timeout на graceful shutdown: 20–30 секунд
- ✓ Зависит от средней длительности запросов
- ✓ Для долгих задач (background jobs) используйте отдельные воркеры и очередь (например, BullMQ)



На что обратить внимание

- Не начинайте новые фоновые задачи
- Закройте все внешние соединения (Redis, MQ и т.д.)
- Обработайте SIGINT (локальная разработка)
- Логируйте все этапы завершения
- Используйте таймаут на принудительное завершение



Хорошее приложение не падает — оно корректно завершается."

Fallback timeout

JAVASCRIPT

```
const forceTimer = setTimeout(() => {  
  logger.fatal("Graceful shutdown timeout");  
  server.closeAllConnections();  
  process.exit(1);  
}, 10000);  
  
forceTimer.unref();
```

Не продолжайте работу после критического состояния

- Uncaught failure может оставить неизвестное state.
- Запишите error и завершите process.
- Надёжный process manager/container должен перезапустить instance.
- Не пытайтесь бесконечно продолжать работу после фатальной ошибки.
- Главная защита — tests, error handling и автоматический restart.



Учебный вопрос 5. Работа за reverse proxy

В production Express часто работает не напрямую с Internet, а за Nginx/load balancer

Reverse proxy

- Принимает внешний HTTP/HTTPS traffic.
- Может завершать TLS/HTTPS.
- Может выполнять compression и caching.
- Может распределять traffic между несколькими instances.
- Передаёт запрос дальше в Node.js/Express.

trust proxy

- Express по умолчанию не доверяет X-Forwarded-* headers.
- За известным reverse proxy можно настроить `app.set("trust proxy", ...)`.
- Настройка влияет на `req.ip`, `req.ips`, `req.hostname` и `req.protocol`.
- Нельзя бездумно ставить `trust proxy=true` в любой topology.
- Policy должна соответствовать реальному числу/адресам proxy hops.

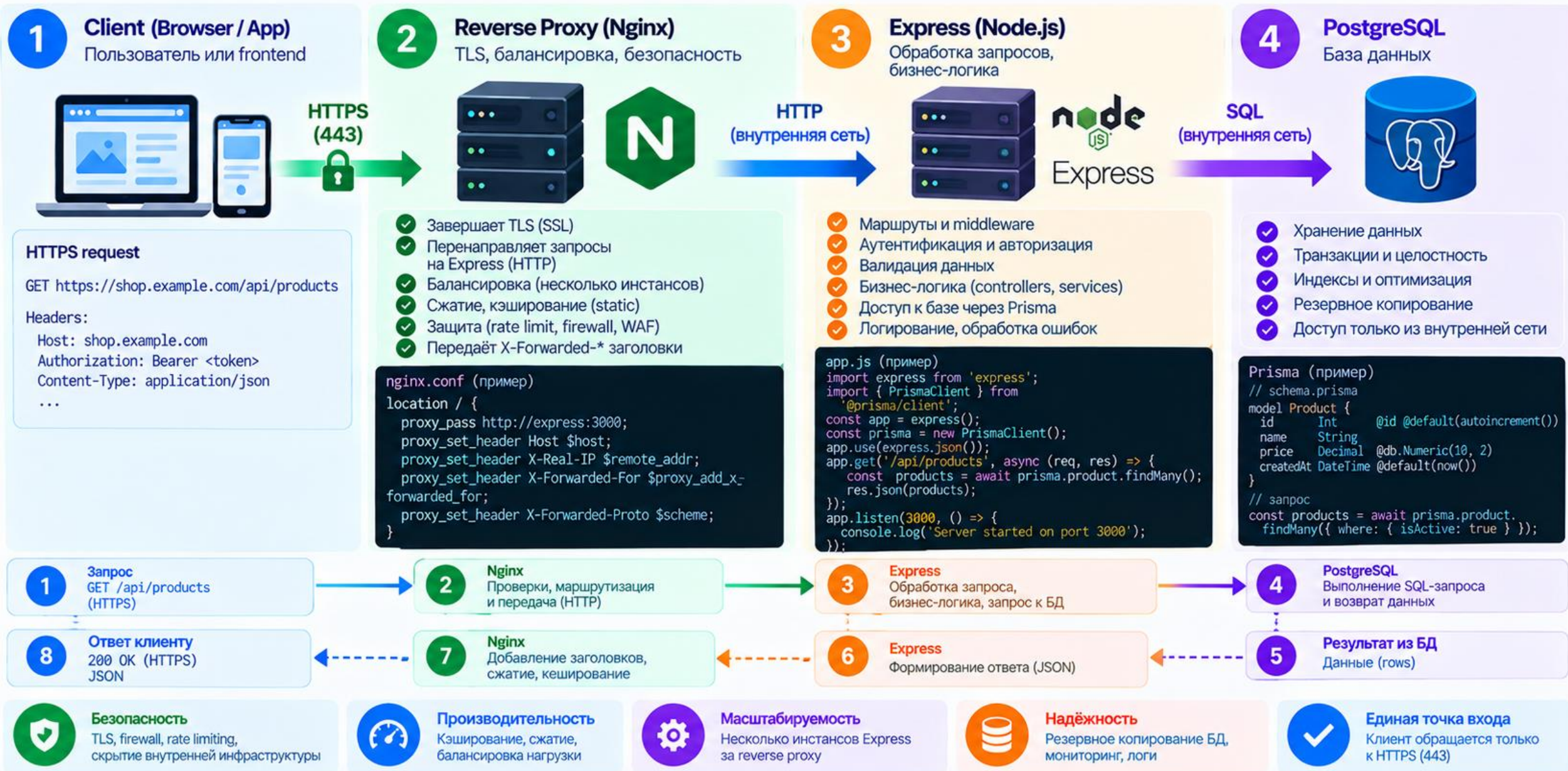
Пример trust proxy

JAVASCRIPT

```
if (env.nodeEnv === "production") {  
  app.set("trust proxy", 1);  
}
```

Production request path: Client → Reverse Proxy → Express → PostgreSQL

Клиент отправляет HTTPS-запрос. Reverse proxy (Nginx) завершает TLS, выполняет балансировку и security-функции, далее передаёт запрос в Express-приложение, которое обращается к базе данных PostgreSQL и возвращает ответ клиенту.



Production-safe error response

- 4xx может содержать безопасное описание клиентской ошибки.
- Unexpected 5xx не должен возвращать stack trace.
- Server log сохраняет техническую Error information.
- Response может возвращать requestId для поддержки.
- Не раскрывайте filesystem paths и database internals.

Production readiness: что проверить до deployment

- NODE_ENV=production.
- Все required environment variables присутствуют.
- Secrets не находятся в Git.
- Health endpoints работают.
- Graceful shutdown проверен.
- Tests проходят.
- Production logs structured и безопасны.
- CORS/Helmet/rate limits настроены.
- Reverse proxy/trust proxy соответствует topology.
- Database migrations подготовлены отдельно от runtime startup.

Мини-практика: production-readiness audit

Подготовить Internet Shop API к безопасному запуску в production environment.

Задание

- Создать config/env.js
- Проверить required variables
- Добавить /health/live
- Добавить /health/ready
- Реализовать SIGTERM/SIGINT shutdown
- Закрывать Prisma
- Добавить shutdown timeout
- Настроить NODE_ENV=production
- Проверить trust proxy policy
- Запустить integration tests

Критерии проверки

- Приложение fail-fast при отсутствии secret
- Health endpoints имеют ожидаемые 200/503
- Ctrl+C завершает server корректно
- Secrets не выводятся в logs
- Integration tests остаются зелёными

Production readiness gate

Приложение должно пройти все этапы, прежде чем попасть в продакшен



Итоги занятия

- 1 Production configuration хранится вне исходного кода.
- 2 Required variables нужно проверять при startup.
- 3 NODE_ENV устанавливается в production.
- 4 app.js отделяется от server.js.
- 5 Liveness проверяет жизнь процесса, readiness — готовность принимать traffic.
- 6 SIGTERM/SIGINT должны запускать graceful shutdown.
- 7 HTTP server и Prisma connections нужно закрывать корректно.
- 8 Reverse proxy часто принимает внешний HTTPS traffic.
- 9 trust proxy должен соответствовать реальной topology.
- 10 Production readiness проверяется до deployment, а не после аварии.